TECHNOVA SOLUTIONS PVT. LTD.

CODE REVIEW GUIDELINES

Document ID	TN-ENG-STD-004
Document Type	Engineering Code Review Guidelines
Document Owner	Engineering
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Guidelines

1. Purpose

These Code Review Guidelines define the expected process and review criteria for evaluating software changes at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”).

The purpose of code review is to improve correctness, maintainability, security, reliability, knowledge sharing, and confidence in changes before they are integrated.

2. Scope

These guidelines apply to code changes submitted through pull requests or other approved review mechanisms, including application code, services, libraries, scripts, infrastructure code, automation, data-processing code, and configuration changes where review is required.

The level of review should be proportional to the risk and impact of the change.

3. Code Review Principles

- Review the change, not the person.
- Focus on correctness, security, maintainability, and business intent.
- Provide specific and constructive feedback.
- Keep reviews timely enough to support delivery.
- Prefer clear evidence and technical reasoning over personal preference.
- Use automated checks for mechanical issues where practical.
- Escalate significant unresolved concerns rather than silently approving them.

4. Responsibilities

Role	Responsibilities
Author	Prepare a reviewable change, explain intent, run required checks, and respond to feedback.
Reviewer	Evaluate correctness, risks, maintainability, security, testing, and relevant project requirements.
Approver	Provide the required approval when sufficiently confident in the change.

Role	Responsibilities
Technical Lead / Owner	Resolve complex technical disagreements and ensure appropriate review depth.
Engineering Manager	Support sustainable review practices, quality expectations, and timely delivery.

5. When Code Review Is Required

Material changes should receive review before integration according to repository requirements.

- New production functionality
- Changes to business logic
- Security-sensitive changes
- Database or schema changes
- API or contract changes
- Infrastructure and deployment changes
- Changes affecting critical services
- Significant refactoring

6. Changes That May Use Reduced Review

Low-risk changes may use a lighter review process where the repository explicitly permits it.

- Typographical corrections
- Documentation-only changes
- Non-functional formatting changes
- Other changes formally classified as low risk

A reduced review process must not be used to bypass required security, access-control, production, or compliance review.

7. Preparing a Change for Review

- Keep the change focused.
- Run the applicable formatter, linter, and tests.
- Review the complete diff before requesting review.
- Remove debugging statements and temporary files.
- Check that no secrets or sensitive data are included.
- Update documentation when behavior or interfaces change.

8. Pull Request Description

The author should provide enough information for a reviewer to understand the change without reconstructing the entire problem from the code.

- Problem or requirement
- Summary of the implementation
- Testing performed
- Important design decisions
- Known risks or limitations

- Deployment, migration, or rollback considerations

9. Review Size

Large changes are harder to review effectively. Authors should divide work into coherent reviewable units when technically practical.

When a large change cannot reasonably be divided, the author should provide additional context and reviewers should consider staged review or additional technical review.

10. Reviewer Selection

Reviewers should have sufficient knowledge of the affected technology, component, business logic, or risk area.

- At least one reviewer familiar with the affected component where practical.
- Additional security, architecture, database, or operational review for high-risk changes when required.
- Avoid selecting reviewers solely because they are available if they lack relevant context.

11. Reviewer Independence

Where repository rules require independent approval, the author should not approve their own change.

For sensitive or high-risk changes, independent review should be performed by an appropriate qualified person.

12. Review Priorities

Reviewers should generally prioritize issues in the following order:

- Security and privacy risks
- Correctness and data integrity
- Reliability and operational impact
- Breaking changes and compatibility
- Performance risks
- Maintainability and design
- Testing adequacy
- Readability and style

13. Correctness

- Does the implementation satisfy the intended requirement?
- Are normal and important edge cases handled?
- Could the change introduce incorrect state or data?
- Are error conditions handled appropriately?
- Does the implementation behave consistently with existing contracts?

14. Business Logic

Reviewers should verify that business rules are implemented accurately and that important assumptions are visible.

- Check boundary conditions.
- Check authorization and eligibility logic.
- Check calculations and transformations.

- Check default behavior.
- Check interactions with existing workflows.

15. Security Review

- Authentication and authorization are appropriate.
- Untrusted input is validated safely.
- Sensitive information is protected.
- Secrets are not exposed.
- Security controls are not accidentally bypassed.
- Dependencies and integrations do not introduce obvious security concerns.

16. Access Control Review

Changes affecting permissions, roles, document access, repositories, or identity should be reviewed for least privilege and authorization correctness.

For enterprise knowledge and AI systems, reviewers should verify that protected documents cannot enter retrieval or response-generation context without authorization.

17. Data and Privacy Review

- Identify new personal or confidential data handling.
- Check whether data collection is necessary.
- Verify access restrictions.
- Avoid unnecessary logging of sensitive information.
- Consider retention and deletion requirements.

18. Error Handling Review

- Unexpected failures are not silently ignored.
- Errors contain useful but non-sensitive diagnostic information.
- Failures do not leave resources or transactions in unsafe states.
- Retry behavior is appropriate.
- User-facing errors do not expose internal details.

19. Performance Review

Reviewers should consider performance when a change affects high-volume, latency-sensitive, or resource-intensive paths.

- Avoid unnecessary database or network calls.
- Consider algorithmic complexity.
- Check unbounded loops, memory growth, or expensive repeated operations.
- Consider caching only when its correctness and invalidation behavior are clear.

20. Database and Data-Model Review

- Schema changes are compatible or include a migration plan.

- Queries are safe and appropriately indexed where necessary.
- Transactions preserve required consistency.
- Large data operations consider operational impact.
- Sensitive fields receive appropriate protection.

21. API and Interface Review

- API behavior matches documented contracts.
- Request and response validation is appropriate.
- Authorization is enforced.
- Backward compatibility is considered.
- Breaking changes are explicitly identified.

22. Maintainability Review

- The code is understandable to another engineer.
- Responsibilities are appropriately separated.
- Abstractions are justified.
- Duplicated complex logic is addressed where appropriate.
- The implementation fits established project patterns.

23. Readability

Code should be understandable without excessive mental effort.

- Use descriptive names.
- Avoid unnecessary nesting.
- Keep control flow clear.
- Prefer consistent patterns.
- Use comments for intent or non-obvious constraints rather than obvious mechanics.

24. Testing Review

Reviewers should evaluate whether the tests provide reasonable confidence in the changed behavior.

- Normal behavior is covered.
- Important edge cases are considered.
- Error paths are tested where relevant.
- Regression tests exist for significant defects.
- Tests are deterministic where practical.

25. Test Quality

A large number of tests does not automatically indicate good coverage. Reviewers should consider whether the tests would fail when important incorrect behavior is introduced.

26. Infrastructure and Configuration Review

Infrastructure-as-code, deployment configuration, permissions, environment variables, and other operational changes should be reviewed for security, reliability, and unintended environment impact.

27. Dependency Review

- Check the purpose of new dependencies.
- Consider maintenance and security status.
- Consider licensing requirements.
- Avoid unnecessary packages.
- Review significant version upgrades for compatibility.

28. AI-Generated Code Review

Code produced with AI assistants or other automated generation tools must receive the same engineering review as manually written code.

- Verify correctness rather than assuming generated code is reliable.
- Check security vulnerabilities and unsafe patterns.
- Review dependency or licensing implications.
- Check for accidental sensitive-data handling.
- Add or update tests as appropriate.

29. Documentation Review

If behavior, interfaces, configuration, architecture, or operational procedures change, reviewers should check that relevant documentation is updated.

30. Review Comments

Review comments should be clear enough for the author to understand the concern and determine an appropriate response.

- Explain the problem and why it matters.
- Distinguish required changes from suggestions.
- Avoid personal or dismissive language.
- Reference standards or evidence when useful.
- Avoid excessive comments on trivial personal preferences.

31. Comment Severity

Level	Meaning
Blocking	The issue must be resolved before approval or requires an explicit authorized decision.
Important	The issue presents meaningful quality, security, maintainability, or operational risk and should normally be addressed.
Suggestion	An optional improvement that may make the implementation clearer or more maintainable.
Question	A request for clarification or additional context; it is not automatically a defect.

32. Reviewer Questions

- What problem is this change solving?

- What assumptions does the implementation make?
- What happens for invalid or unexpected input?
- What happens if a dependency is unavailable?
- Could this change expose sensitive information?
- Could it break an existing consumer?
- Is the change observable in production?
- Can the change be safely rolled back?
- Are the tests sufficient for the risk?

33. Author Response

Authors should respond to material review comments and either implement the requested change, explain the technical reasoning, or obtain an appropriate decision when disagreement remains.

Resolved comments should not be silently reopened unless new evidence or a related concern arises.

34. Technical Disagreements

Technical disagreements should be resolved using requirements, evidence, established standards, experiments, measurements, or architectural principles.

If the disagreement cannot be resolved between author and reviewer, it should be escalated to the relevant technical lead, architect, or component owner.

35. Review Turnaround

Teams should aim to review changes promptly to avoid unnecessary delivery delays.

Authors should provide sufficient context and identify urgent or blocking delivery requirements without pressuring reviewers to ignore material concerns.

36. Review Fatigue

Teams should avoid unnecessary review overhead.

- Keep pull requests focused.
- Use automated tools for formatting and mechanical checks.
- Do not require multiple reviewers for trivial changes without a risk-based reason.
- Use additional reviewers when the change genuinely warrants deeper expertise.

37. Approval

An approval indicates that the reviewer has evaluated the change to the level reasonably expected for their role and has no unresolved blocking concerns within the scope of the review.

Approval does not transfer ownership of the code from the author to the reviewer.

38. Changes After Approval

Material changes made after approval may require renewed review.

- Significant logic changes
- New files or components
- Security-sensitive modifications

- Major conflict resolution changes
- Changes that invalidate the reviewer's original assessment

39. Stale Approvals

Repository settings may invalidate approvals when a pull request changes substantially. Teams should follow the configured repository behavior and obtain renewed approval where appropriate.

40. Automated Review Tools

Static analysis, linters, security scanners, dependency checks, formatting tools, and AI-assisted review tools may supplement human review.

Automated tools do not replace human judgment for business logic, architecture, security context, or important operational decisions.

41. Review of Database Migrations

- Check forward migration behavior.
- Consider rollback or recovery.
- Consider data volume and locking.
- Check compatibility between old and new application versions where rolling deployments are used.
- Verify sensitive data is handled appropriately.

42. Review of Deployment Changes

Deployment-related changes should be reviewed for environment impact, permissions, configuration, rollback, observability, and service availability.

Detailed deployment requirements are defined in the Deployment Policy.

43. Review of Critical Systems

Changes affecting critical services may require additional review, testing, or approval based on system risk.

Criticality should be considered when determining review depth rather than applying identical review effort to every change.

44. Emergency and Hotfix Reviews

Emergency fixes may use an expedited review process when required to reduce immediate production or security risk.

- Keep the change narrowly focused.
- Obtain the highest-priority feasible review.
- Perform appropriate validation.
- Document the emergency nature of the change.
- Conduct follow-up review where necessary.

45. Review Records

Pull requests, approvals, comments, automated checks, and merge history may form part of the engineering record for a change.

Records should be retained according to applicable repository and records-management requirements.

46. Confidentiality During Review

Code reviews must not expose confidential source code, credentials, customer data, employee information, or other restricted information to unauthorized persons.

Review discussions should use approved repositories and collaboration channels.

47. Security Issues Found During Review

A reviewer who identifies a potentially serious security vulnerability should handle it according to Information Security procedures and avoid unnecessarily exposing exploit details in broadly visible review discussions.

48. Review Anti-Patterns

- Approving without reading the change.
- Focusing only on formatting while ignoring correctness.
- Blocking a change solely because of personal style preferences.
- Making vague comments such as “fix this” without explanation.
- Using review as a substitute for requirements clarification.
- Allowing urgency to eliminate necessary security review.
- Performing an enormous review after changes have accumulated for weeks.

49. Code Review Checklist

Area	Reviewer Check
Purpose	Is the intent of the change clear?
Correctness	Does the implementation satisfy requirements and edge cases?
Security	Are authentication, authorization, secrets, and input risks addressed?
Data	Are privacy, integrity, and sensitive-data concerns addressed?
Performance	Are important resource or latency risks considered?
Testing	Is the changed behavior appropriately tested?
Maintainability	Is the implementation understandable and appropriately structured?
Compatibility	Could existing consumers or integrations break?
Operations	Are logging, monitoring, deployment, and rollback considerations addressed?
Documentation	Are relevant documents or interfaces updated?

50. Frequently Asked Questions

Does every line need a comment?

No. Comments should explain intent, reasoning, constraints, or non-obvious behavior rather than restating clear code.

Can I approve my own pull request?

Not where independent approval is required by the repository or project.

What if I disagree with a reviewer?

Discuss the technical reasoning and evidence. Escalate to the appropriate technical lead or owner when necessary.

Can AI-generated code bypass review?

No. AI-generated code is subject to the same review and validation expectations.

Should reviewers check every style issue manually?

Mechanical style issues should preferably be handled by formatters and automated checks so human review can focus on higher-value concerns.

What if a pull request is too large?

Split it into coherent reviewable units when practical, or provide additional context and use staged review.

Can emergency fixes skip review?

They may use expedited review when necessary, but should remain traceable and receive the highest feasible validation.

When should an approval be renewed?

When material changes after approval invalidate the original review.

Who owns the code after approval?

The responsible engineering team and code owner retain ownership; review approval does not transfer responsibility.

What should I do if I find a serious security issue?

Follow the appropriate Information Security process and avoid unnecessarily exposing sensitive details.

51. Guidelines Administration

Engineering owns these Code Review Guidelines and reviews them at least annually or when material changes occur in development practices, repository tooling, security requirements, or organizational needs.

The latest approved version maintained in the official Company repository is the authoritative version.

52. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Code Review Guidelines	Engineering	VP Engineering / CTO

53. Related Documents

- Engineering Handbook
- Coding Standards
- Git Workflow
- Deployment Policy
- Testing Standards
- Information Security Policy
- Access Control Policy

END OF DOCUMENT

TN-ENG-STD-004 | Version 1.0 | Internal — Engineering | Engineering